

C++ Programming

A Practical Reference Guide

Core Language, Memory Management, Object-Oriented Programming,
Templates, Exception Handling, File I/O, and the Standard Template
Library (STL)

Compiled Reference Edition

Table of Contents

1. Introduction to C++
2. Setting Up and Compiling C++ Code
3. Variables, Data Types, and Operators
4. Strings and Input/Output
5. Arrays, Vectors, and Containers
6. Control Flow: Conditionals and Loops
7. Functions and Overloading
8. Pointers, References, and Memory Management
9. Object-Oriented Programming
10. Templates and Generic Programming
11. Exception Handling
12. File Input and Output
13. The Standard Template Library (STL)
14. Modern C++ and Best Practices
15. Quick Reference Tables

1. Introduction to C++

C++ is a general-purpose, compiled programming language created by Bjarne Stroustrup as an extension of the C language, with the first version released in 1985. It combines low-level control over memory and hardware with high-level abstractions such as classes, templates, and exceptions, making it suitable for systems programming, game engines, embedded software, and performance-critical applications.

1.1 Key Characteristics

- **Compiled:** C++ source code is translated ahead of time into native machine code by a compiler, producing fast executables.
- **Statically typed:** Every variable's type is known at compile time, which catches many errors before the program ever runs.
- **Multi-paradigm:** C++ supports procedural, object-oriented, generic, and to some extent functional programming styles.
- **Manual and RAII-based resource management:** Programmers can manage memory directly with pointers, or rely on RAII and smart pointers for automatic cleanup.
- **Zero-overhead abstractions:** Language features are designed so that you do not pay a performance cost for abstractions you do not use.

1.2 Philosophy

C++'s design philosophy favors giving the programmer maximum control and efficiency, trusting them to use powerful features correctly. This is often summarized as "you don't pay for what you don't use." Modern C++ (C++11 and later) has added many safety-oriented features such as smart pointers, range-based loops, and move semantics, while retaining full backward compatibility with C-style code.

2. Setting Up and Compiling C++ Code

2.1 Installing a Compiler

Common C++ compilers include GCC (g++) on Linux, Clang on macOS and Linux, and MSVC on Windows. Most Linux distributions include g++ in their package manager, and Xcode Command Line Tools provide Clang on macOS. After installation, verify the version from a terminal:

```
$ g++ --version
g++ (Ubuntu 13.2.0) 13.2.0
```

2.2 Compiling and Running a Program

Unlike interpreted languages, C++ source files must be compiled into an executable before they can run. A minimal program looks like this:

```
// hello.cpp
#include <iostream>

int main() {
    std::cout << "Hello, World!" << std::endl;
    return 0;
}

// Compile with: g++ -std=c++17 -o hello hello.cpp
// Run with:     ./hello
// Output:      Hello, World!
```

2.3 The Build Process

Compiling a C++ program happens in stages: preprocessing (expanding #include and #define directives), compilation (translating source into object code), and linking (combining object files and libraries into a final executable). For multi-file projects, build tools such as Make or CMake automate this process.

```
# A minimal CMakeLists.txt
cmake_minimum_required(VERSION 3.10)
project(MyApp)
set(CMAKE_CXX_STANDARD 17)
add_executable(myapp main.cpp)

# Build:
$ mkdir build && cd build
$ cmake ..
$ make
```

Note: Always compile with warnings enabled, e.g. `-Wall -Wextra`, to catch common mistakes such as uninitialized variables or signed/unsigned comparisons.

3. Variables, Data Types, and Operators

3.1 Declaring Variables

Every variable in C++ has a fixed type, declared before use. C++11 introduced `auto` for type inference from the initializer.

```
int age = 30;
double price = 19.99;
char grade = 'A';
bool is_valid = true;
std::string name = "Alice";

// Type inference
auto count = 10;           // int
auto ratio = 3.14;         // double
```

3.2 Fundamental Data Types

Type	Description	Typical Size
int	Signed integer	4 bytes
short	Short signed integer	2 bytes
long	Long signed integer	8 bytes
float	Single-precision floating point	4 bytes
double	Double-precision floating point	8 bytes
char	Single character	1 byte
bool	Boolean true/false	1 byte
std::string	Sequence of characters	variable

3.3 Constants

```
const double PI = 3.14159;
constexpr int MAX_SIZE = 100;    // evaluated at compile time
```

3.4 Operators

- **Arithmetic:** + - * / %
- **Comparison:** == != < > <= >=
- **Logical:** && || !
- **Assignment:** = += -= *= /=

- **Increment/decrement:** ++ --
- **Bitwise:** & | ^ ~ << >>

4. Strings and Input/Output

4.1 The `std::string` Class

The C++ standard library provides `std::string` for working with text, defined in the `<string>` header. It manages its own memory and supports concatenation, comparison, and many utility methods.

```
#include <string>

std::string first = "Hello";
std::string full = first + ", World!";
std::cout << full.length() << std::endl;    // 13
std::cout << full.substr(0, 5) << std::endl; // "Hello"
full.append("!!!");
bool has = full.find("World") != std::string::npos;
```

4.2 Console Input and Output

The `<iostream>` header provides `std::cin` for input and `std::cout` for output, connected via the stream insertion (`<<`) and extraction (`>>`) operators.

```
#include <iostream>

std::string name;
int age;
std::cout << "Enter your name: ";
std::cin >> name;
std::cout << "Enter your age: ";
std::cin >> age;
std::cout << name << " is " << age << " years old.\n";
```

4.3 Formatted Output

```
#include <iomanip>

double pi = 3.14159265;
std::cout << std::fixed << std::setprecision(2) << pi << "\n"; // 3.14
std::cout << std::setw(10) << "right" << "\n";
```

5. Arrays, Vectors, and Containers

5.1 C-Style Arrays

C++ inherits fixed-size arrays from C. Their size must be known at compile time and they do not know their own length at runtime.

```
int numbers[5] = {1, 2, 3, 4, 5};
std::cout << numbers[0] << std::endl;    // 1
int size = sizeof(numbers) / sizeof(numbers[0]);    // 5
```

5.2 `std::vector`

`std::vector`, from the `<vector>` header, is a dynamic array that grows and shrinks automatically and is the default choice for sequential storage in modern C++.

```
#include <vector>

std::vector<int> nums = {1, 2, 3};
nums.push_back(4);
nums.pop_back();
std::cout << nums.size() << std::endl;

for (int n : nums) {
    std::cout << n << " ";
}
```

5.3 Other Standard Containers

Container	Header	Use case
<code>std::array</code>	<code><array></code>	Fixed-size array with STL interface
<code>std::vector</code>	<code><vector></code>	Dynamic, resizable array
<code>std::deque</code>	<code><deque></code>	Double-ended queue
<code>std::list</code>	<code><list></code>	Doubly linked list
<code>std::set</code>	<code><set></code>	Sorted, unique elements
<code>std::map</code>	<code><map></code>	Sorted key-value pairs
<code>std::unordered_map</code>	<code><unordered_map></code>	Hash-table key-value pairs

6. Control Flow: Conditionals and Loops

6.1 if / else if / else

```
int score = 85;
if (score >= 90) {
    std::cout << "A";
} else if (score >= 80) {
    std::cout << "B";
} else {
    std::cout << "C or below";
}
```

6.2 switch Statements

```
int day = 3;
switch (day) {
    case 1:
        std::cout << "Monday";
        break;
    case 2:
        std::cout << "Tuesday";
        break;
    default:
        std::cout << "Another day";
}
```

6.3 Loops

```
// for loop
for (int i = 0; i < 5; ++i) {
    std::cout << i << " ";
}

// range-based for loop (C++11+)
std::vector<int> nums = {10, 20, 30};
for (int n : nums) {
    std::cout << n << " ";
}

// while loop
int count = 0;
while (count < 3) {
    std::cout << count;
    ++count;
}

// do-while loop
int n = 0;
do {
    std::cout << n;
    ++n;
} while (n < 3);
```

Note: Use `break` to exit a loop early and `continue` to skip to the next iteration.

7. Functions and Overloading

7.1 Defining Functions

A C++ function declares the type of every parameter and its return value. Functions must be declared or defined before they are used, unless a prototype is provided earlier in the file.

```
int add(int a, int b) {
    return a + b;
}

int main() {
    int result = add(2, 3);
    std::cout << result << std::endl; // 5
}
```

7.2 Default Arguments

```
double power(double base, int exponent = 2) {
    double result = 1.0;
    for (int i = 0; i < exponent; ++i) result *= base;
    return result;
}

power(3.0);          // 9.0 (exponent defaults to 2)
power(2.0, 5);       // 32.0
```

7.3 Function Overloading

C++ allows multiple functions with the same name as long as their parameter lists differ, a feature called overloading. The compiler selects the correct version based on the arguments provided.

```
int multiply(int a, int b) { return a * b; }
double multiply(double a, double b) { return a * b; }

multiply(2, 3);          // calls the int version
multiply(2.5, 4.0);     // calls the double version
```

7.4 Pass by Value vs. Pass by Reference

```
void increment(int x) { x++; }           // copy, caller unaffected
void incrementRef(int& x) { x++; }       // reference, caller changed
void printVec(const std::vector<int>& v) { // const ref avoids copying
    for (int n : v) std::cout << n << " ";
}
```

8. Pointers, References, and Memory Management

8.1 Pointers

A pointer holds the memory address of another variable. Pointers are declared with an asterisk and dereferenced with the same symbol to access the value they point to.

```
int x = 42;
int* ptr = &x;           // ptr holds the address of x
std::cout << *ptr;       // 42, dereference to get the value
*ptr = 100;               // modifies x through the pointer
std::cout << x;           // 100
```

8.2 References

A reference is an alias for an existing variable. Unlike a pointer, it must be initialized when declared and cannot be reseated to refer to something else afterward.

```
int a = 10;
int& ref = a;
ref = 20;
std::cout << a;    // 20
```

8.3 Dynamic Memory

Memory allocated with `new` lives on the heap until explicitly released with `delete`. Forgetting to delete causes a memory leak.

```
int* p = new int(5);
std::cout << *p;
delete p;

int* arr = new int[10];
delete[] arr;
```

8.4 Smart Pointers

Modern C++ discourages raw `new/delete` in favor of smart pointers from `<memory>`, which free memory automatically when they go out of scope (RAII).

```
#include <memory>

std::unique_ptr<int> up = std::make_unique<int>(42);
std::shared_ptr<int> sp = std::make_shared<int>(7);
auto sp2 = sp;    // shared ownership, ref count is now 2
```

Note: Prefer `std::unique_ptr` when only one owner is needed, and `std::shared_ptr` only when shared ownership is genuinely required, since it has reference-counting overhead.

9. Object-Oriented Programming

9.1 Classes and Objects

```
class BankAccount {
private:
    std::string owner;
    double balance;

public:
    BankAccount(std::string owner, double balance = 0.0)
        : owner(owner), balance(balance) {}

    void deposit(double amount) {
        balance += amount;
    }

    double getBalance() const {
        return balance;
    }
};

BankAccount acc("Alice", 100.0);
acc.deposit(50.0);
std::cout << acc.getBalance();    // 150
```

9.2 Access Specifiers

- **public:** accessible from anywhere the object is visible.
- **protected:** accessible within the class and its derived classes.
- **private:** accessible only within the class itself (the default for `class`).

9.3 Inheritance

```
class Animal {
public:
    virtual void speak() const {
        std::cout << "Some generic sound\n";
    }
    virtual ~Animal() = default;
};

class Dog : public Animal {
public:
    void speak() const override {
        std::cout << "Woof!\n";
    }
};

Animal* a = new Dog();
a->speak();    // "Woof!" (polymorphism via virtual functions)
delete a;
```

9.4 Operator Overloading

```
class Point {  
public:  
    double x, y;  
    Point(double x, double y) : x(x), y(y) {}  
  
    Point operator+(const Point& other) const {  
        return Point(x + other.x, y + other.y);  
    }  
};  
  
Point p1(1, 2), p2(3, 4);  
Point p3 = p1 + p2;    // (4, 6)
```

10. Templates and Generic Programming

10.1 Function Templates

Templates let you write a single function or class that works with many types, with the compiler generating a specific version for each type actually used.

```
template <typename T>
T maxValue(T a, T b) {
    return (a > b) ? a : b;
}

maxValue(3, 7);           // works with int
maxValue(2.5, 1.1);       // works with double
maxValue(std::string("a"), std::string("b")); // works with string
```

10.2 Class Templates

```
template <typename T>
class Box {
private:
    T value;
public:
    Box(T value) : value(value) {}
    T get() const { return value; }
};

Box<int> intBox(42);
Box<std::string> strBox("hello");
```

10.3 Standard Template Library Roots

Templates are the foundation of the STL: containers like `std::vector<T>` and algorithms like `std::sort` are all generic, working with any type that supports the required operations.

11. Exception Handling

11.1 try, catch, and throw

C++ handles runtime errors using exceptions. Code that might fail is wrapped in a `try` block, and errors are raised with `throw` and handled in one or more `catch` blocks.

```
#include <stdexcept>

double divide(double a, double b) {
    if (b == 0) {
        throw std::invalid_argument("Division by zero");
    }
    return a / b;
}

try {
    double result = divide(10, 0);
} catch (const std::invalid_argument& e) {
    std::cout << "Error: " << e.what() << std::endl;
} catch (...) {
    std::cout << "Unknown error" << std::endl;
}
```

11.2 Standard Exception Types

Exception	Meaning
<code>std::runtime_error</code>	General runtime failure
<code>std::logic_error</code>	Error in program logic
<code>std::invalid_argument</code>	An argument does not have a valid value
<code>std::out_of_range</code>	Access outside a valid range (e.g. <code>.at()</code>)
<code>std::bad_alloc</code>	Memory allocation failed

Note: Always catch exceptions by reference (`const std::exception&`) to avoid slicing, and prefer RAI so that resources are released automatically even when an exception unwinds the stack.

12. File Input and Output

12.1 Writing to a File

```
#include <fstream>

std::ofstream outFile("data.txt");
if (outFile.is_open()) {
    outFile << "Hello, file!\n";
    outFile << 42 << std::endl;
    outFile.close();
}
```

12.2 Reading from a File

```
#include <fstream>
#include <string>

std::ifstream inFile("data.txt");
std::string line;
while (std::getline(inFile, line)) {
    std::cout << line << std::endl;
}
inFile.close();
```

12.3 File Modes

```
std::ofstream f1("out.txt", std::ios::app);    // append
std::ofstream f2("out.bin", std::ios::binary); // binary mode
std::fstream f3("data.txt", std::ios::in | std::ios::out);
```

Note: File streams close automatically when they go out of scope thanks to RAII, but calling `close()` explicitly makes intent clear and surfaces errors earlier.

13. The Standard Template Library (STL)

13.1 Algorithms

The `<algorithm>` header provides generic functions that operate on ranges defined by iterators, working across nearly any container type.

```
#include <algorithm>
#include <vector>

std::vector<int> v = {5, 3, 1, 4, 2};
std::sort(v.begin(), v.end());           // 1 2 3 4 5
auto it = std::find(v.begin(), v.end(), 3); // iterator to 3
int total = std::accumulate(v.begin(), v.end(), 0); // 15
bool any = std::any_of(v.begin(), v.end(),
    [](int x) { return x > 4; });
```

13.2 Iterators

Iterators generalize the idea of a pointer walking through a container, letting the same algorithm work on vectors, lists, sets, and more.

```
std::vector<int> v = {10, 20, 30};
for (auto it = v.begin(); it != v.end(); ++it) {
    std::cout << *it << " ";
}
```

13.3 `std::map` and `std::set`

```
#include <map>

std::map<std::string, int> ages;
ages["Alice"] = 30;
ages["Bob"] = 25;
for (const auto& [name, age] : ages) {
    std::cout << name << ": " << age << "\n";
}

#include <set>
std::set<int> uniqueNums = {3, 1, 2, 1, 3}; // {1, 2, 3}
```

13.4 Lambda Expressions

```
auto square = [](int x) { return x * x; };
std::cout << square(5); // 25

int threshold = 3;
auto isAbove = [threshold](int x) { return x > threshold; }; // capture
```

14. Modern C++ and Best Practices

Since C++11, the language has evolved rapidly, with C++14, C++17, and C++20 adding features that make code safer and more expressive while preserving performance.

14.1 Move Semantics

```
std::vector<int> makeVector() {  
    std::vector<int> v = {1, 2, 3};  
    return v;    // moved, not copied, when returned  
}  
  
std::vector<int> a = makeVector();  
std::vector<int> b = std::move(a);    // explicit move, a is now empty
```

14.2 Structured Bindings (C++17)

```
std::pair<int, std::string> p = {1, "one"};  
auto [id, name] = p;  
std::cout << id << " " << name;
```

14.3 Style Guidelines

- Prefer `const` and references over copying large objects.
- Prefer smart pointers over raw `new/delete`.
- Initialize variables at the point of declaration.
- Use `nullptr` instead of `NULL` or `0` for null pointers.
- Mark member functions `const` when they do not modify the object.
- Follow the Rule of Zero: let compiler-generated special member functions do the work whenever possible instead of writing your own.

15. Quick Reference Tables

15.1 Common Standard Library Headers

Header	Purpose
<iostream>	Console input/output
<string>	std::string class
<vector>	Dynamic array container
<map> / <unordered_map>	Key-value containers
<algorithm>	sort, find, accumulate, etc.
<memory>	Smart pointers
<fstream>	File input/output streams
<stdexcept>	Standard exception classes

15.2 Compilation Flags Cheat Sheet

```
g++ -std=c++17 file.cpp -o app      # choose language standard
g++ -Wall -Wextra file.cpp -o app   # enable useful warnings
g++ -O2 file.cpp -o app              # optimize for speed
g++ -g file.cpp -o app               # include debug symbols
```

15.3 Common STL Algorithm Patterns

```
std::sort(v.begin(), v.end());
std::reverse(v.begin(), v.end());
std::max_element(v.begin(), v.end());
std::count(v.begin(), v.end(), value);
std::unique(v.begin(), v.end());
std::transform(v.begin(), v.end(), v.begin(),
               [](int x) { return x * 2; });
```

This guide is a condensed, original reference summarizing core C++ concepts and is not a reproduction of the official C++ standard or cppreference.com. For complete and authoritative details, consult cppreference.com.